

반도체 파운드리 대시보드 AI 확장 제안 문서

기존 매출·생산·품질 대시보드에 "문서 검색까지 되는" AI 챗봇을 얹는 작업의 제안요청서, 기술제안서, 프로그램 아키텍처 설계서를 하나로 묶었습니다. 기술 용어는 최소화하고, 그림으로 먼저 이해되도록 구성했습니다.

DOC SET RFP · PROPOSAL · ARCHITECTURE PROJECT foundry-dashboard DEMO 2026-07-30

STATUS 구현 · 로컬 검증 완료

개요 제안요청서 배경 및 목적 현황 (AS-IS) 요구사항 제약사항 **기술제안서** 제안 개요 전체 구성도

이 문서는 무엇인가

이미 운영 중인 반도체 파운드리 매출·생산·품질 대시보드(Spring Boot + PostgreSQL + Vue)에, 사내 문서(출장 규정, 품질 매뉴얼 등)까지 자연어로 물어볼 수 있는 기능을 추가했습니다. 아래 세 문서는 순서대로 읽으면 됩니다 — 무엇을 요청받았는지 → 어떻게 풀기로 했는지 → 실제로 어떻게 만들어졌는지 순서입니다.

한 줄 요약 — 기존 챗봇은 "숫자(매출·수율·불량)"만 답할 수 있었습니다. 이번에 "문서(규정·매뉴얼)"도 같은 창구에서 답할 수 있도록, 별도 서비스 하나를 새로 붙이고 챗봇이 질문 성격에 따라 자동으로 갈라 타도록 만들었습니다.

01 제안요청서 (RFP)

무엇이, 왜 필요했는지에 대한 정리입니다.

1.1 배경 및 목적

현재 대시보드는 매출·생산·품질처럼 **표로 정리된 데이터**는 잘 보여주지만, "출장비는 얼마까지 인정되나요?", "CRITICAL 불량이 나오면 어떻게 처리하나요?" 같은 **문서 속 규정**에는 답하지 못했습니다. 같은 챗봇 창구에서 두 종류 질문을 모두 처리할 수 있게 하는 것이 이번 작업의 목적입니다.

1.2 현황 (AS-IS)

구성 요소	현재 상태
Backend	Spring Boot 3.2 (Java 21) + JdbcTemplate, PostgreSQL 직접 조회
Database	PostgreSQL 16 — 고객/제품/주문/생산Lot/불량 5개 테이블
Frontend	Vue 3 + Chart.js, KPI 카드·차트·표 대시보드
AI 기능	자연어 → SQL 변환 챗봇 (Claude 직접 연동, SELECT 전용 가드레일)
배포	Railway, Docker 단일 서비스

1.3 요구사항

ID	요구사항	우선순위	상태
R1	사내 문서를 올려두고 자연어로 질의응답	필수	완료
R2	정형 데이터 질문 / 문서 질문을 자동으로 구분	필수	완료
R3	문서 답변에는 출처(파일명)와 신뢰도 표시	필수	완료
R4	업로드한 문서를 목록으로 보고 삭제 가능	필수	완료
R5	기존 대시보드·SQL 챗봇 기능은 그대로 유지	필수	완료
R6	로컬에서 스크립트 한 번으로 즉시 데모 실행	필수	완료
R7	외부 API 키가 없어도 데모가 끊기지 않아야 함	권장	완료
R8	운영(Railway) 서버에도 동일 기능 배포	선택	후속 작업

1.4 제약사항

- 데모 목적이므로 로그인·권한 체계는 만들지 않고, CORS만 열어 둔다.
- 기존 운영 배포(Railway)에 영향을 주지 않는 방식으로 작업한다.
- 기존 데이터베이스(PostgreSQL)를 그대로 사용하고, 테이블만 추가한다.

02 기술제안서

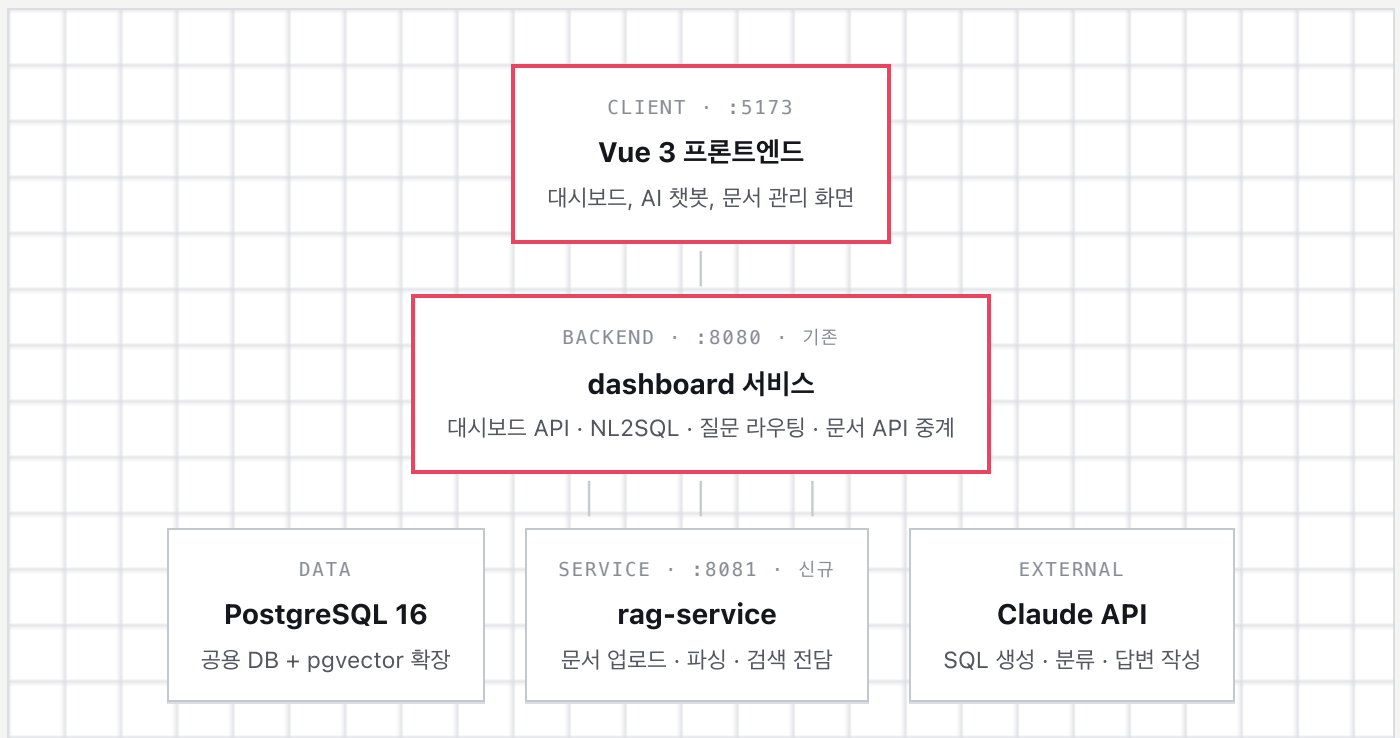
위 요구사항을 어떤 방식으로 풀기로 했는지에 대한 설명입니다.

2.1 제안 개요

기존 챗봇(정형 데이터용)은 그대로 두고, **문서 검색을 전담하는 서비스 하나를 새로 만들어 옆에 붙였습니다.** 챗봇은 질문을 받으면 "이건 숫자를 물어보는 질문인지, 문서 내용을 물어보는 질문인지"를 먼저 스스로 판단한 뒤, 맞는 쪽으로 자동으로 넘겨서 답을 만듭니다. 사용자 입장에서는 창구가 하나뿐이라 구분해서 물어볼 필요가 없습니다.

2.2 전체 구성도

같은 데이터베이스를 공유하는 두 개의 서버(기존 백엔드 + 신규 문서검색 서비스)와, 화면(Vue) 하나로 구성됩니다.



점선처럼 보이는 세로선은 실제로는 HTTP 호출입니다. 화면은 백엔드 하나만 알면 되고, rag-service는 백엔드가 대신 호출해 주므로 외부에는 드러나지 않습니다.

2.3 기존 대비 무엇을 추가했나

기존 (AS-IS)

- Spring Boot + JdbcTemplate
- PostgreSQL 5개 테이블
- Vue 3 + Chart.js 대시보드
- NL2SQL 챗봇 (숫자 질문만)

추가한 것

- pgvector (문서 유사도 검색)
- Apache Tika (문서 텍스트 추출)
- 임베딩 서비스 (OpenAI, 키 없으면 자체 폴백)
- rag-service 모듈 + 문서 관리 화면

결과 (TO-BE)

- 숫자 질문·문서 질문 모두 답하는 챗봇
- 출처·신뢰도가 표시되는 답변
- 업로드 문서 관리(목록/삭제)
- 기존 화면·기능은 그대로

2.4 핵심 기능 — 실제 예시로 보기

사용자 질문 입력 (창구는 하나)

▼ 자동 분류 ▼

예) "수익이 가장 높은 고객은?"

1. 숫자 질문으로 분류
2. 기존 NL2SQL 경로로 처리 (변경 없음)
3. SELECT 전용 가드레일 통과 후 조회

예) "사내 출장 규정 요약해줘"

1. 문서 질문으로 분류

4. 결과를 한국어로 요약해 답변	2. rag-service에 검색 요청 (신규)
	3. 의미가 가장 가까운 문서 조각을 찾음
	4. 그 내용을 근거로 답변 + 출처·신뢰도 표시

- 출처 표시 — 답변 아래에 어떤 파일의 몇 번째 조각을 근거로 썼는지, 유사도(%)까지 함께 보여줍니다.
- 문서 업로드 — 챗봇 옆  버튼으로 즉시 업로드하면 바로 검색 대상에 포함됩니다.
- 문서 관리 화면 — 지금까지 올라간 문서를 한눈에 보고, 필요 없어진 문서는 삭제할 수 있습니다.

2.5 진행 상황

작업	내용	상태
구조 설계	멀티모듈 구성(backend + rag-service), 역할 분리	완료
RAG 파이프라인	업로드→파싱→칭킹→임베딩→저장→검색	완료
질문 자동 라우팅	SQL / 문서 질문 자동 분류	완료
문서 관리 화면	목록 조회 + 삭제	완료
로컬 통합 검증	SQL 질문·문서 질문·가드레일·업로드·삭제 전 구간 테스트 (2026-07-19)	완료
운영(Railway) 반영	rag-service를 운영 배포에 추가	미착수
실문서 업로드 리허설	실제 규정·매뉴얼 PDF로 데모 사전 점검	권장

2.6 리스크와 대응

리스크	대응
OpenAI 키 미설정 시 문서 검색 정확도 저하	어휘 기반 폴백으로 데모는 끊기지 않음. 정확도를 위해 데모 전 키 설정 권장
로컬 Postgres에 pgvector 미지원	이미 소스 빌드로 설치·검증 완료. Docker 데모 경로는 애초에 영향 없음
운영 서버 미반영	이번 범위는 로컬 데모로 한정. 필요 시 후속 작업으로 진행
대용량 문서 업로드 지연	데모용 업로드 용량 20MB로 제한

03 프로그램 아키텍처 설계서

실제로 어떻게 만들어졌는지에 대한 상세 설명입니다.

3.1 모듈 구성

모듈	포트	책임
backend	8080	대시보드 API, NL2SQL 가드레일, 질문 라우팅, 문서 API 중계
rag-service	8081	문서 업로드, 파싱(Tika), 청킹, 임베딩, pgvector 검색·삭제
frontend	5173	대시보드 화면, AI 챗봇, 문서 관리 화면 — backend만 호출

두 서버는 같은 저장소(Git) 안에서 `pom.xml` 하나로 함께 관리되는 멀티모듈 구조입니다. backend의 배포 방식은 건드리지 않도록, 최상위 pom은 두 모듈을 묶어주지만 하고 각자 독립적으로도 빌드되게 설계했습니다.

3.2 질문 처리 흐름

- STEP 1

질문 접수
사용자가 챗봇에 질문을 입력하면 backend의 `/api/ai/chat`이 받습니다.
- STEP 2

자동 분류
Claude에 짧은 분류 요청을 보내 "SQL" 또는 "DOCUMENT"로 구분합니다.
- STEP 3

분기 처리
아래 두 경로 중 하나로 진행합니다 (기존 SQL 경로는 변경 없음).

SQL 경로	문서 경로
1. Claude가 질문을 SELECT 쿼리로 변환 2. 가드레일 검증 (DDL/DML/위험명령 차단, LIMIT 강제)	1. backend → rag-service에 검색 요청

3. PostgreSQL 조회

4. Claude가 결과를 한국어로 요약

2. 질문을 임베딩으로 변환

3. pgvector 코사인 유사도로 상위 청크 조회

4. Claude가 청크를 근거로 답변 + 출처 생성

3.3 문서 인입(업로드) 파이프라인



1

업로드

Vue → backend(중계) → rag-service로 파일 전달



2

텍스트 추출

Apache Tika로 PDF/DOCX/TXT 등에서 순수 텍스트만 뽑아냄



3

청킹

문단 단위로 약 800자씩, 앞 조각과 10%를 겹치게 잘라 문맥이 끊기지 않게 함



4

임베딩

각 조각을 숫자 벡터로 변환 (OpenAI, 키 없으면 자체 풀백 방식)



5

저장

조각 + 벡터를 pgvector에 저장하고 문서 상태를 완료로 갱신

3.4 데이터베이스 스키마

테이블	구분	설명
customers / products / sales_orders / production_lots / defect_records	기 존	고객·제품·주문·생산Lot·불량 (변경 없음)
document_meta	신 규	업로드 문서의 파일명·유형·크기·청크 수·상태
document_chunks	신 규	문서 조각 본문 + 임베딩 벡터(1536차원, pgvector) + ivfflat 인덱스

3.5 API 목록

메서드	경로	구분	설명
GET	/api/kpis 외 6종	기존	대시보드 KPI·차트·표 데이터
POST	/api/chat	기존 유지	단순 문자열 응답 (하위호환)
POST	/api/ai/chat	신규	구조화 응답 {answer, type, sources, confidence}
POST	/api/documents/upload	신규	문서 업로드 (rag-service로 중계)
GET	/api/documents	신규	업로드 문서 목록
DELETE	/api/documents/{'id'}}	신규	문서 및 체크 삭제

3.6 SQL 가드레일 (기존 유지)

- SELECT 문만 허용, 그 외(DDL/DML)는 정규식으로 차단
- CREATE·DROP·ALTER·TRUNCATE 등 스키마 변경 명령 차단
- INSERT·UPDATE·DELETE 등 데이터 변경 명령 차단
- GRANT·EXEC·COPY 등 위험 명령 차단
- 세미콜론으로 이어진 복수 문장 차단
- 결과는 최대 100건으로 자동 제한(LIMIT)

3.7 배포 구성

환경	구성	비고
로컬 (docker-compose)	db(pgvector 이미지) + backend + rag-service + frontend	완료
로컬 (start.sh)	Homebrew Postgres 16 + 세 프로세스 순차 기동	완료
운영 (Railway)	backend + frontend만 배포 중, rag-service 미반영	후속 작업

부록 · 용어정리

비전공자를 위한 최소한의 용어 설명입니다.

RAG

질문과 관련된 문서 조각을 먼저 찾은 뒤, 그 내용을 근거로 AI가 답변을 만드는 방식.

NL2SQL

자연어 질문을 데이터베이스 조회문(SQL)으로 자동 변환하는 방식.

pgvector

PostgreSQL에서 벡터(임베딩)를 저장하고 "의미가 비슷한 것"을 빠르게 찾게 해주는 확장 기능.

임베딩

글의 의미를 숫자들의 목록(벡터)으로 표현한 것. 의미가 비슷하면 벡터도 서로 가까워진다.

청킹

긴 문서를 검색하기 좋은 크기(문단 단위, 약 800자)로 잘게 나누는 작업.

가드레일

AI가 데이터 삭제 등 위험한 동작을 하지 못하도록 막아주는 안전장치.